

Benchmark

LoReflection-Eval: 面向家具布局生成的可用性感知诊断式评估基准

LoReflection-Eval 是一个面向室内家具布局生成与编辑任务的可用性感知诊断式评估基准。基于 3D-FRONT / 3D-FUTURE 的场景和家具资产构建，保留传统 scene synthesis 评估中的对象完整性、分布质量、文本遵循、关系满足、碰撞和越界等指标，同时进一步引入 **Practical Usability**（作为我们论文的亮点），用于评估生成布局是否具备真实居住场景中的基本使用可行性。

现有 indoor scene synthesis 评估通常关注 FID / KID、对象类别与数量、关系满足率、collision 和 out-of-bound 等指标。这些指标是必要的，但它们不能完整判断一个家具布局是否真正好用。一个布局可能没有家具重叠，也没有越出房间边界，但仍然可能因为餐桌离墙太近、衣柜没有开门空间、沙发与电视距离不合理等问题而难以实际使用。LoReflection-Eval 的核心补充正是 **Practical Usability**：判断布局是否不仅看起来合理、几何合法，而且能够作为真实室内空间布置被使用。

Benchmark Package

对于每个场景，我们构造一个 benchmark package：

代码块

```
1  S_i = {  
2    A_i: architecture annotation,  
3    Y_i: ground-truth object layout,  
4    P_i: paired text descriptions,  
5    C_i: evaluation constraint specification  
6  }
```

S_i 不是要求所有方法接收的统一输入协议，而是同一个底层场景的多视图评估包。不同方法仍然使用各自的 native input protocol；评估时，所有方法的输出都会被转换到同一个方法无关标准的 evaluation representation，再由同一套统一评估流程计算指标。

A_i 表示从场景中提取或派生的建筑信息，包括 room type、room boundary，以及可用的 door / window anchors。这里的建筑信息主要服务于统一评估，而不是要求所有 baseline 都接收完全相同的建筑输入。

Y_i 表示真实对象级布局，包括家具类别、中心位置、尺寸、朝向、footprint 和可选 asset reference。

P_i 表示与该场景配对的不同粒度文本描述。coarse prompt 描述房间类型和主要家具类别；medium prompt 进一步描述常见空间关系；fine prompt 进一步加入方向、距离、门窗和 clearance 相关约束。

C_i 表示该场景的评估约束规格说明，即这个场景需要检查哪些对象、关系、几何规则和可用性规则。它是评估器计算指标时读取的检查标准，而不是最终评价分数本身。

代码块

```
1  C_i = {
2      required_objects: [...],
3      required_relations: [...],
4      functional_relations: [...],
5      geometric_rules: [00B, Collision, DoorWindowBlocking],
6      usability_rules: [...]
7  }
```

其中 **required_objects** 和 **required_relations** 来自文本描述；**functional_relations** 来自房间类型和家具组合先验；**geometric_rules** 是无条件硬几何规则；**usability_rules** 根据场景中存在的家具类别和建筑锚点自动激活。

这个 package 不假设 architecture condition 和 text condition 的信息量完全等价。它只保证这些条件来自同一个 ground-truth scene，并最终在同一个方法无关的标准评估表示上进行评价。

标准 Evaluation Representation

所有方法输出统一转换为 evaluation representation：

代码块

```
1  {
2      room_id,
3      room_type,
4      room_boundary,
5      architectural_anchors,
6      furniture_instances: [
7          {
8              category,
9              center,
10             size,
11             orientation,
12             footprint,
13             optional_asset_id
14         }
15     ]
16 }
```

`coordinate transform` 用于将 3D-FRONT 世界坐标、俯视布局平面和语义图像像素坐标对齐，使不同方法输出能够在同一尺度下计算 footprint、distance、collision、OOB 和 door / window blocking。

核心评估维度

LoReflection-Eval 的主体包含四个核心评估维度：

代码块

- 1 Scene Quality
- 2 Semantic Compliance
- 3 Geometric Validity
- 4 Practical Usability

其中 **Scene Quality**、**Semantic Compliance** 和 **Geometric Validity** 保证与已有 scene synthesis / text-conditioned scene generation 评估保持可比性；**Practical Usability** 是 LoReflection-Eval 的核心扩展，用于补充现有指标难以覆盖的真实使用问题。

Semantic Compliance（语义符合度） 内部进一步区分 **Text Alignment** 和 **Functional Relation**。Text Alignment 评估文本中显式要求是否被满足；Functional Relation 评估即使文本没有明确说，布局作为某类房间是否满足基本功能关系。

Scene Quality

Scene Quality 衡量生成布局是否包含正确的家具类别和数量，以及整体分布是否接近真实数据。

指标包括：

代码块

- 1 Object Count F1 ↑
- 2 FID ↓
- 3 KID ↓

Object Count F1 衡量生成布局是否包含正确的家具类别和数量，同时惩罚 missing objects 和 extra objects。

FID / KID 用于衡量生成场景分布与真实 3D-FRONT 分布的接近程度，作为 dataset-level 的分布真实感指标报告。FID / KID 不作为布局合规性和可用性的唯一依据。

指标层级：

代码块

- 1 Scene-level:
- 2 Object Count F1
- 3
- 4 Dataset-level:
- 5 FID
- 6 KID

Semantic Compliance

Semantic Compliance 衡量生成布局是否满足文本中的显式要求，以及是否符合房间类型和家具组合所隐含的功能关系。

Semantic Compliance 包含两个子部分：

代码块

- 1 Text Alignment
- 2 Functional Relation

Text Alignment

Text Alignment 衡量生成布局是否遵循输入文本中的显式要求。

指标包括：

代码块

- 1 Required Object SR ↑
- 2 Required Relation SR ↑

Required Object SR 检查文本中明确要求的对象是否被生成。它只用于 text-conditioned setting，关注 prompt 中显式提到的对象是否出现。

Required Relation SR 检查文本明确要求的空间关系是否满足，例如 “desk near window” 或 “cabinet left of dining table” 。

Required Object SR 和 Required Relation SR 根据输入文本粒度确定适用范围。coarse prompt 只描述房间类型和家具类别，因此仅计算 Required Object SR；medium / fine prompt 包含空间关系描述，因此同时计算 Required Relation SR。

Text Alignment 评估的是 instruction following 能力。它回答的问题是：模型是否听懂并执行了文本里明确说出的要求。

Instruction Satisfaction 不作为 benchmark 核心指标。它可以通过 VLM judge 或 human evaluation 作为 supplementary metric 报告，但不进入主自动评估指标。若需要自动化整体文本满足度，可以由 Required Object SR 和 Required Relation SR 组合得到：

代码块

```
1  Instruction Satisfaction_auto =  
2   $\alpha$  · Required Object SR +  $\beta$  · Required Relation SR
```

其中 α 和 β 根据文本中对象要求与关系要求的数量动态确定。

Functional Relation

Functional Relation 衡量生成布局是否满足房间类型和家具组合所隐含的功能关系。

指标包括：

代码块

```
1  Functional Relation SR ↑
```

Functional Relation SR 包含两类关系。

第一类是 furniture-furniture functional relations，例如：

代码块

```
1  nightstand beside bed  
2  sofa facing TV  
3  chairs around dining table
```

第二类是 furniture-architecture functional relations，例如：

代码块

```
1  desk near window  
2  bed against wall
```

Functional Relation 与 Text Alignment 不同。Text Alignment 评估文本中显式要求是否被满足；Functional Relation 评估即使文本没有明确说，布局作为某类房间是否满足基本功能关系。

这种拆分可以区分两类失败模式：如果 Required Relation SR 低但 Functional Relation SR 高，说明模型具有一定布局常识，但没有很好遵循指令；如果 Required Relation SR 高但 Functional Relation SR 低，说明模型执行了局部指令，但整体功能布局不合理。

Geometric Validity

Geometric Validity 衡量布局是否违反无条件硬几何约束。该维度只保留明确的硬错误，不再单独设置宽泛的 Clearance Violation 指标，以避免和 Practical Usability 中的 Opening Clearance / Circulation Clearance 重叠。

指标包括：

代码块

1

OOB ↓

2

Collision ↓

3

Door / Window Blocking ↓

OOB 衡量家具 footprint 是否越出房间边界。

Collision 衡量家具之间是否发生重叠。

Door / Window Blocking 衡量家具是否直接阻挡门的开启区域或严重遮挡窗户区域。这是无条件硬错误：无论文本是否提到门窗，只要门被阻挡或窗户被严重遮挡，都视为建筑几何违规。

其中，门阻挡检查家具 footprint 是否侵入门扇扫过区域或门开启所需的硬性操作区域；窗阻挡检查家具是否严重遮挡窗户开口区域，例如遮挡比例超过硬性阈值。该指标是 binary hard violation。

Geometric Validity 与 Practical Usability 的区别是：Geometric Validity 关注绝对不能发生的硬错误，例如越界、碰撞、门扇被阻挡、窗户被严重遮挡；Practical Usability 关注使用尺度是否合理，例如门打开后门洞附近是否仍有足够通行净宽、窗前是否保留基本采光 / 通风 / 操作空间、衣柜是否有开合空间、通道是否足够宽。

因此，原本宽泛的 Clearance Violation 不作为核心独立指标保留。门窗硬性阻断归入 Door / Window Blocking；门窗附近通行净宽、窗前可用空间、家具开合净空和通行动线净空归入 Practical Usability。

Practical Usability

Practical Usability 是 LoReflection-Eval 的核心扩展。它衡量一个几何合法的布局是否具备真实居住场景中的基本使用可行性。

一个布局即使没有碰撞、没有越界，也可能因为以下问题而不可用：

- 1 餐桌离墙太近，椅子无法正常拉出；
- 2 衣柜离床太近，柜门或抽屉无法打开；
- 3 书桌后方缺少入座空间；
- 4 沙发与电视距离过近或过远；
- 5 床侧通道太窄，无法正常行走和整理床铺。

Practical Usability 的主指标为：

代码块

```
1 Practical Usability SR =  
2 # satisfied applicable usability rules  
3 /  
4 # applicable usability rules
```

同时报告：

代码块

```
1 Usability Violation Rate =  
2 # scenes with at least one violated usability rule  
3 /  
4 # evaluated scenes
```

Usability Violation Rate 是 Practical Usability SR 的场景级汇总形式，用于直观显示有多少场景至少存在一个可用性问题，而不是一个独立的新评价维度。

Practical Usability 由三个子维度组成：

代码块

```
1 Functional Distance SR ↑  
2 Opening Clearance SR ↑  
3 Circulation Clearance SR ↑
```

Functional Distance SR 检查常见家具组合之间是否具有可使用距离。

Opening Clearance SR 检查门、窗和家具开合区域是否保留足够操作空间。

Circulation Clearance SR 检查主要通行动线和局部活动空间是否可用。

Practical Usability 不替代 OOB、Collision 或 Door / Window Blocking，而是补充它们无法覆盖的“几何合法但实际不好用”的情况。

Practical Usability 规则池

为保证规则池简洁、可复现，并与 3D-FRONT 类别稳定兼容，Practical Usability 采用 12 条候选规则，分为 3 个子维度，每个子维度 4 条。

Functional Distance

代码块

- 1 FD-1: Sofa-TV viewing distance
- 2 FD-2: Sofa-CoffeeTable reach distance
- 3 FD-3: Bed-Nightstand reach distance
- 4 FD-4: Desk-Chair seating distance

Sofa-TV viewing distance 衡量沙发与电视柜 / 电视区域之间是否具有合理观看距离。

Sofa-CoffeeTable reach distance 衡量沙发与茶几之间是否具有可触达但不拥挤的使用距离。

Bed-Nightstand reach distance 衡量床与床头柜之间是否保持合理贴近关系。

Desk-Chair seating distance 衡量书桌与椅子之间是否有基本坐姿使用距离。

Opening Clearance

代码块

- 1 OC-1: Doorway passage clearance
- 2 OC-2: Window access clearance
- 3 OC-3: Wardrobe opening clearance
- 4 OC-4: Bed-Window clearance

Doorway passage clearance 检查门完全打开后，门洞附近是否仍保留足够通行净宽，使人员可以正常进出房间。它不同于 Door / Window Blocking 中的门扇弧区硬阻挡：后者检查门是否能打开，前者检查门打开后是否仍有可用通行空间。

Window access clearance 检查窗前是否保留足够采光、通风和操作空间。它不同于 Door / Window Blocking 中的窗户硬遮挡：后者检查窗户开口是否被严重遮挡，前者评估窗前区域的可用性是否因家具布局而降低。

Wardrobe opening clearance 检查衣柜前方是否有足够开合和取物空间。

Bed-Window clearance 检查床是否过于贴近窗户，导致采光、窗帘或开窗使用受限。

Circulation Clearance

代码块

- 1 CC-1: Main passage clearance
- 2 CC-2: Bed-side passage clearance
- 3 CC-3: Dining circulation clearance
- 4 CC-4: Desk rear clearance

Main passage clearance 检查主要通行动线是否保留足够通行宽度。

Bed-side passage clearance 检查床侧是否保留基本通行和整理床铺空间。

Dining circulation clearance 检查餐桌周围是否保留入座、起身和绕行空间。

Desk rear clearance 检查书桌后方是否保留椅子后退和入座空间。

Practical Usability 阈值校准

Practical Usability 的数值阈值不直接手工指定。我们首先从常见室内空间可用性需求中构造候选约束池，然后在 3D-FRONT training split 上进行校准。

校准流程如下：

代码块

- 1 candidate rule construction
- 2 → category mapping
- 3 → applicable sample mining
- 4 → measurement extraction
- 5 → train-set distribution statistics
- 6 → rule filtering
- 7 → threshold fixing

Category mapping 将 3D-FRONT / 3D-FUTURE 的对象类别映射到评估类别，例如 sofa、coffee table、TV stand、dining table、chair、bed、nightstand、wardrobe、desk、cabinet、shelf。

Applicable sample mining 只在对应家具类别或建筑锚点存在时激活规则。例如，Sofa-TV viewing distance 只在场景中同时存在 sofa 和 TV stand 时评估；Wardrobe opening clearance 只在存在 wardrobe 且能够估计其操作侧时评估；Doorway passage clearance 只在 door anchor 可用时评估。

Measurement extraction 从生成布局或真实布局的 object footprint、room boundary 和 architectural anchors 中计算距离、clearance 和 free-space regions。这里的 free-space regions 是评估时由 room boundary 减去 furniture footprints 后推导得到的可通行区域，不是 3D-FRONT 的原生字段，也不是额外输入。

Train-set distribution statistics 统计训练集上的 mean、median、standard deviation、P5、P10、P25、P75、P90、P95，以及候选阈值下的 satisfaction rate。

Rule filtering 只保留满足以下条件的规则：

代码块

- 1 在训练集中具有足够 applicable samples;
- 2 测量失败率低;
- 3 在 ground-truth layouts 中具有较高满足率;
- 4 能够稳定映射到 3D-FRONT 类别和建筑标注;

Threshold fixing 在 training split 上完成。之后 validation / test split 以及所有生成方法的输出都使用同一组固定阈值，不再根据测试结果调整。

指标聚合方式

Practical Usability SR 按 applicable rules 聚合：

代码块

```
1 Practical Usability SR =  
2 # satisfied applicable rules  
3 /  
4 # applicable rules
```

三个子维度分别计算：

代码块

```
1 Functional Distance SR =  
2 # satisfied FD rules  
3 /  
4 # applicable FD rules  
5  
6 Opening Clearance SR =  
7 # satisfied OC rules  
8 /  
9 # applicable OC rules  
10  
11 Circulation Clearance SR =  
12 # satisfied CC rules  
13 /  
14 # applicable CC rules
```

Usability Violation Rate 按场景聚合：

代码块

```
1 Usability Violation Rate =  
2 # scenes with at least one violated usability rule  
3 /  
4 # evaluated scenes
```

主文报告 Practical Usability SR 和 Usability Violation Rate。三个子维度的 breakdown 放入附录。

Editing Extension

对于 **language-guided layout editing** 任务，LoReflection-Eval 在核心 benchmark 指标之外，额外报告少量 editing-specific metrics。编辑任务与生成任务不同，它不是从零生成一个布局，而是在已有 source layout 上执行局部修改。因此，评估需要回答三个编辑特有的问题：

代码块

- 1 1. 编辑命令是否完成？
- 2 2. 是否编辑了正确目标对象？
- 3 3. 非目标对象是否被保持？

因此，Editing Extension 只定义三个编辑专属指标：

代码块

- 1 Edit Success ↑
- 2 Target Localization ↑
- 3 Non-target Preservation ↑

Edit Success 衡量编辑命令是否被成功执行。不同 edit type 采用对应判定方式：translate 检查目标对象是否按指定方向 / 距离移动；rotate 检查朝向变化是否满足命令；scale 检查尺寸变化是否正确；replace 检查类别或资产语义是否被替换；add 检查目标对象是否被添加；remove 检查目标对象是否被删除。

Target Localization 衡量编辑是否作用在正确目标对象上。例如指令为 “move the desk near the window” 时，系统应移动 desk，而不是移动 chair、bed 或其他非目标对象。对于 replace、remove、rotate 和 scale 等操作，Target Localization 尤其重要，因为目标对象错误时，即使最终场景看起来合理，也不能算编辑成功。

Non-target Preservation 衡量 protected non-target objects 是否保持稳定。对于不应被编辑的对象，比较 source layout 和 edited layout 中的 category、center position、size、orientation 和 footprint IoU。对象满足以下条件时视为 preserved：

```
代码块
1 category unchanged
2 center drift <  $\tau_{\text{pos}}$ 
3 size change <  $\tau_{\text{size}}$ 
4 rotation drift <  $\tau_{\text{rot}}$ 
5 footprint IoU >  $\tau_{\text{iou}}$ 
```

除这三个编辑专属指标外，编辑后的布局仍然使用 LoReflection-Eval 的核心指标进行评价，包括 Semantic Compliance、Geometric Validity 和 Practical Usability。也就是说，编辑实验不重新定义一套新的布局质量指标，而是在 edited layout 上复用核心 benchmark 指标，检查编辑后场景是否仍然满足功能关系、几何合法性和实际可用性。

为了衡量编辑是否引入新的副作用，我们额外报告 **Edit-induced Hard Error**：

```
代码块
1 Edit-induced Hard Error ↓
```

它统计编辑后相对于 source layout 新增的硬错误，包括 new collision、new OOB 和 new door / window blocking。

language-guided editing 的主表报告：

```
代码块
1 Edit Success ↑
2 Target Localization ↑
3 Non-target Preservation ↑
4 Edit-induced Hard Error ↓
5 Functional Relation SR ↑
6 Practical Usability SR ↑
```

其中 Functional Relation SR 和 Practical Usability SR 直接在 edited layout 上计算，属于 LoReflection-Eval 核心指标的复用；Edit-induced Hard Error 用于诊断编辑过程是否引入新的硬错误。

LoReflection-specific Diagnostics

这组指标不属于 LoReflection-Eval 的核心 benchmark 维度。只用于分析具有中间修复状态的方法，主要对应 **Table 3: Ablation on Reflection, Review, and Repair Components** 和 **Efficiency / Convergence Analysis**。

在 **Table 3** 中，我们使用以下指标评估闭环修复是否真正有效：

```
代码块
1 Target Resolved ↑
2 Hard Violation Reduction ↑
3 Relation Issue Reduction ↑
4 New Hard Error ↓
5 Non-target Preservation ↑
```

Target Resolved 衡量被诊断的目标问题是否被成功修复。例如 missing nightstand 是否被补全，desk not near window 是否被修正，wardrobe blocking door 是否被解除。

Hard Violation Reduction 衡量修复前后硬几何错误的减少比例，包括 collision、OOB 和 door / window blocking 等。

Relation Issue Reduction 衡量修复前后关系错误的减少比例，包括 not beside、not near、wrong facing、not against wall 等。

New Hard Error 衡量修复过程中是否引入新的硬错误。例如原本只是缺少 nightstand，修复后虽然补上了 nightstand，但新产生了 collision、OOB 或 door / window blocking。

Non-target Preservation 衡量非目标对象是否保持稳定，包括 category unchanged、center shift、size change、rotation drift 和 footprint IoU。

在 **Efficiency / Convergence Analysis** 中，我们进一步报告：

```
代码块
1 Repair Efficiency ↓
2 Average Accepted Rounds ↓
3 Wall-clock Time ↓
4 Cumulative Token Count ↓
5 Cumulative Inpainting Calls ↓
```

Repair Efficiency 定义为首次达到 `target resolved = 1` 且 `hard violation = 0` 所需的修复轮数，并设置最大轮数上限。若在最大轮数内未达到该条件，则记为失败或记为上限值。

Average Accepted Rounds 衡量系统在 early stopping 下实际接受的平均修复轮数。

Wall-clock Time 衡量从输入房型 / 指令到输出最终布局的端到端耗时。

Cumulative Token Count 衡量闭环过程中 VLM 调用累计消耗的 prompt / completion tokens。

Cumulative Inpainting Calls 衡量每个样本实际调用局部修复 / inpainting 模块的次数。

这些指标主要用于解释 LoReflection 或其他具有迭代修复 / 反思过程的方法。

Final Metric Organization

LoReflection-Eval 的核心评估维度为：

代码块

```
1 Scene Quality:
2   Object Count F1
3   FID / KID
4
5   Semantic Compliance:
6     Required Object SR
7     Required Relation SR
8     Functional Relation SR
9
10  Geometric Validity:
11    OOB
12    Collision
13    Door / Window Blocking
14
15  Practical Usability:
16    Practical Usability SR
17    Usability Violation Rate
```

任务扩展指标为：

代码块

```
1   Editing Extension:
2     Edit Success
3     Target Localization
4     Non-target Preservation
5     Functional Relation SR after Edit
6     Edit-induced Collision
7     Edit-induced OOB
8     Edit-induced Door / Window Blocking
9     Practical Usability SR after Edit
```

方法分析指标为：

代码块

```
1   LoReflection-specific Diagnostics:
2     Target Resolved
3     Hard Violation Reduction
4     Relation Issue Reduction
5     New Hard Error
6     Non-target Preservation
7
8   Efficiency / Convergence:
9     Repair Efficiency
10    Average Accepted Rounds
```

- 11 Wall-clock Time
- 12 Cumulative Token Count
- 13 Cumulative Inpainting Calls